Neural Governor: RL-Driven CPU Scheduling for Power-Thermal Optimization in ARM big.LITTLE

Siva Subramanyam Ghanta, and Ganesh Reddy Karri

***Abstract*—The existing mobile operating system CPU schedulers are fairly heuristic, such as the Window-Assisted Load Tracking (WALT) governor. Such algorithms utilize aggressive, reactive boosting so as to provide responsiveness in the user interface. These models have low latency, but result in severe frequency oscillation, resulting in non-proportional dynamic power draw and thermal throttling at sustained load. In this paper, Neural Governor is introduced, a dynamic CPU governor, which leverages Reinforcement Learning, in the form of Q-learning that natively executes on the Qualcomm Snapdragon 8s Gen 3 chip. Neural Governor anticipates and forecasts requirements when the workload needs to be scheduled by modeling Dynamic Voltage and Frequency Scaling (DVFS) as a Markov Decision Process rather than by workload gauging that is already applied. Neural Governor demonstrated a 17.7% power reduction of the system as systems playback 720p 60fps of media, and that the system reached absolute thermal equilibrium. In addition to this, it also reduced the scheduler overhead by 37.0% when there was heavy multitasking. The data have achieved these gains with a marginal burst latency cost of 45.8ms, a mathematically justified method of showing the feasibility of machine learning-based scheduling as compared to old-fashioned heuristics.**

***Index Terms*—Reinforcement Learning, CPU Scheduling, DVFS, ARM big.LITTLE, Thermal Optimization, WALT**

I. Introduction

The main issue regarding modern design of the System-on-Chip (SoC) is how to find a compromise between the maximum computational rate, thermal limits on the chip, and passive cooling of the mobile chassis. In ARM big.LITTLE architectures, the CPU governor is the brain of the power management subsystem, and decides the Voltage and Frequency (V/f) state of each independent cluster of cores.

Schedulers in Linux kernels supplied by Android consumer allocations follow an emphasis on responsiveness of the user interface graphically on long-lasting electrical performance instead of electrical efficiency. Other Governors such as WALT use immediate throttle of the Prime performance core to maximum frequency, in response to abrupt bursts of load. Even though this guarantees fluid application launches; it yields a very poor power profile with constant frequency spikes and in kernel development this is referred to as hunting. Any shift to a higher level of voltages not directly needed is an exponential cost to power, and any surplus heat, putting the SoC closer to its thermal trip point.

S. S. Ghanta is with the department of Computer Science and Engineering (AI/ML), Vellore Institute of Technology, AP, Vijayawada, India (e-mail: ghsivasubramanyam@gmail.com).

G. R. Karri is with the Department of SCOPE, Vellore Institute of Technology, AP, Vijayawada, India (e-mail: ganesh.reddy@vitap.ac.in).

Neural Governor suggests a radical break with such reactive and fixed-point heuristics. The governor is trained on how to find the optimum V/f conditions with a natural application of an anticipatory Reinforcement Learning algorithm on the Linux kernel, with past state-action rewards. This paper will explain the architecture, mathematical concepts and practical validation of the Neural Governor in four different extremes of operation. Specifically, our core contributions are:

- **MDP Formulation with Thermal Inertia:** Modeled CPU scheduling as a Markov Decision Process that actively projects future thermal momentum to prevent reactive throttling.
- **Ultra-Lightweight Kernel Implementation:** Engineered a 1.6 KB quantized Q-table that executes completely within the L1 cache at bare-metal memory speeds.
- **Zero-Overhead CFS Integration:** Decoupled the RL matrix evaluation from the Linux Completely Fair Scheduler, achieving pure $\mathcal{O}(1)$ execution without spinlock gridlock.
- **Empirical Silicon Validation:** Demonstrated a 17.7% power reduction and a 37.0% decrease in scheduler overhead natively on a commercial Snapdragon 8s Gen 3 processor against the WALT governor.

II. Background, Operating System Internals

To appreciate why Neural Governor is an essential development, a decomposition of the Linux CPUFreq subsystem, and operations of the Completely Fair Scheduler (CFS) is required.

The Linux kernel has developed more than threshold-based governors to Energy Aware Scheduling (EAS). Per-Entity Load Tracking (PELT) and WALT were introduced by EAS as a means of gaining a better understanding of asymmetric clusters. WALT is a CPU utilization calculation method that determines the CPU utilization over discrete time periods, generally 16-20 milliseconds. The frequency is scaled up by the governor when utilization goes beyond a hardcoded limit in a window. WALT has the fatal flaw of being retrospective in that it is only able to forecast load by examining the past. This results in severe overshoot—the CPU runs to a huge V/f value just in time when the computation is complete, and before the voltage change expends the energy used to change its value.

Moreover, stock Android kernels use an unnatural ”touch-boost” logic which is part of the input framework. Upon a touchscreen interrupt, the scheduler puts the Prime core automatically to full speed. This hard coded panic behavior guarantees frame rates of animations work well on UI, but essentially cripples power efficiency of the SoC.

Earlier attempts of integrating Machine Learning into the kernel space have been unsuccessful due to the computation cost. System-wide lag Deep Neural Networks (DNNs) are based on matrix multiplications with spinlocks, preventing the Completely Fair Scheduler (CFS) from completing context switches. Neural Governor uses a Q-table approach that is highly optimized with $\mathcal{O}(1)$ look-up, ensuring the runtime kernel scheduler is never blocked on floating-point computation.

III. Derivation and Formulation of Mathematic Framework and MDP

Neural Governor formulates the problem of DVFS as a Markov Decision Process (MDP). An interaction between the agent and the kernel environment occurs at every time step (as outlined in the architectural flow in Fig. 1).

A. Dynamic Power and Voltage Penalty

The CMOS relationship between the activity factor (α), capacitance (C), voltage (V), and frequency (f) determines the dynamic power consumed by a CPU:

$$P_{dynamic} = \alpha \cdot C \cdot V^2 \cdot f \tag{1}$$

Since the voltage required to operate the power supply should be greater as the clock frequency goes up, the power consumption is cubically proportional during aggressive boost phases of WALT.

B. The Stability Metric

We use the Coefficient of Variation (CV) to measure the hunting behaviour of WALT compared to the stability behavior of Neural Governor, because the former measures the ratio of the standard deviation (σ) to the mean (μ):

$$CV = \frac{\sigma}{\mu} \tag{2}$$

When the standard deviation of the CPU frequency is large, it means that there is a wasted voltage transition.

C. Reward System and Effectiveness

The State Space is based on low-latency kernel telemetry which interrogates the actual use of each cluster, the current system temperature, and the actual power draw at the Power Management IC. Each polling period, the agent will choose an action to move the Operating Performance Point (OPP) either up, down or hold.

The Reward Function is designed in such a way as to penalize the cubic power penalty of states of high voltages. Massive indemnification of the agent is awarded against actualization of high system efficacy (η_{sys}), which can be characterized as total computational throughput ($\bar{I}_{perf}$) divided by average wattage ($\bar{P}_{sys}$):

$$\eta_{sys} = \frac{\bar{I}_{perf}}{\bar{P}_{sys}} \tag{3}$$

It is severely penalized against violating thermal limits (ΔT) and having a high Coefficient of Variation (CV_{freq}). The reward at time t is formulated as:

$$R_t = (w_1 \cdot \eta_{sys}) - (w_2 \cdot \Delta T) - (w_3 \cdot CV_{freq}) \tag{4}$$

D. Q-Table Convergence and Hyperparameter Optimization

Although the theoretical use of Q-learning can be applied to the problem of CPU scheduling without complex knowledge, tuning of hyperparameters is necessary to implement it in a live operating system. A model that learns too slowly cannot adjust to new workloads of its users; a model that learns too fast starts to rewrite the states which look stable and degenerates into the very frequency oscillation which we are attempting to avoid.

It was necessary to ensure the learning rate (α) was kept as low as possible. Background tasks in a mobile setting often burst to observe a couple of microseconds in an anomaly. A high learning rate would cause the Q-table to quickly learn to memorise these anomalies and corrupt the logic of the governor. Neural Governor enforces a low learning rate, which makes the workload in Neural Governor scale-consistent across multiple execution windows, before the agent changes the scale up plan of its long-term strategies.

The Q-learning update rule operates continuously in kernel space via the Bellman equation:

$$Q(s_t, a_t) \leftarrow (1-\alpha)Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) \right] \tag{5}$$

The most important variable as far as thermal management is concerned is the discount factor (γ). This measure determines the extent to which the artificial intelligence will be concerned with future rewards as compared to immediate rewards. There are heuristics in stock control, such as WALT, that essentially have a discount of zero: they are merely concerned with how fast they can process the current frame. Neural Governor makes use of a high discount factor. The model mathematically identifies the fact that although a boosting to maximum frequency will give a high immediate throughput reward; will lead to an enormous thermal penalty within a very short period of time. The AI penalises the most to learn that given a five minute window; a medium, stable frequency is associated with the best cumulative reward.

In the early stage of training on the device, the governor used an ϵ-greedy exploration approach. It sometimes defied its own reasoning to experiment with its random CPU frequencies, and to find out the behaviour of the physical hardware with varying voltages. To optimize convergence, an exponential decay function was applied to the exploration rate:

$$\epsilon(t) = \epsilon_{min} + (\epsilon_{max} - \epsilon_{min}) \cdot e^{-\lambda t} \tag{6}$$

The exploration rate went to the ground, and the model switched to pure exploitation as the Q-table filled up with valid data, becoming trapped in the highly optimized scaling paths of the model.

E. Thermal Inertia State Projection

Modern SoCs have very high Thermal Inertia—the inertia of heat generation that remains in the silicon die even after frequency decreases. The governors of yesteryear are blind, and only react when the absolute thermal limits have been violated.

Neural Governor combines thermal momentum with its state perception. Instead of mapping the state to the present absolute

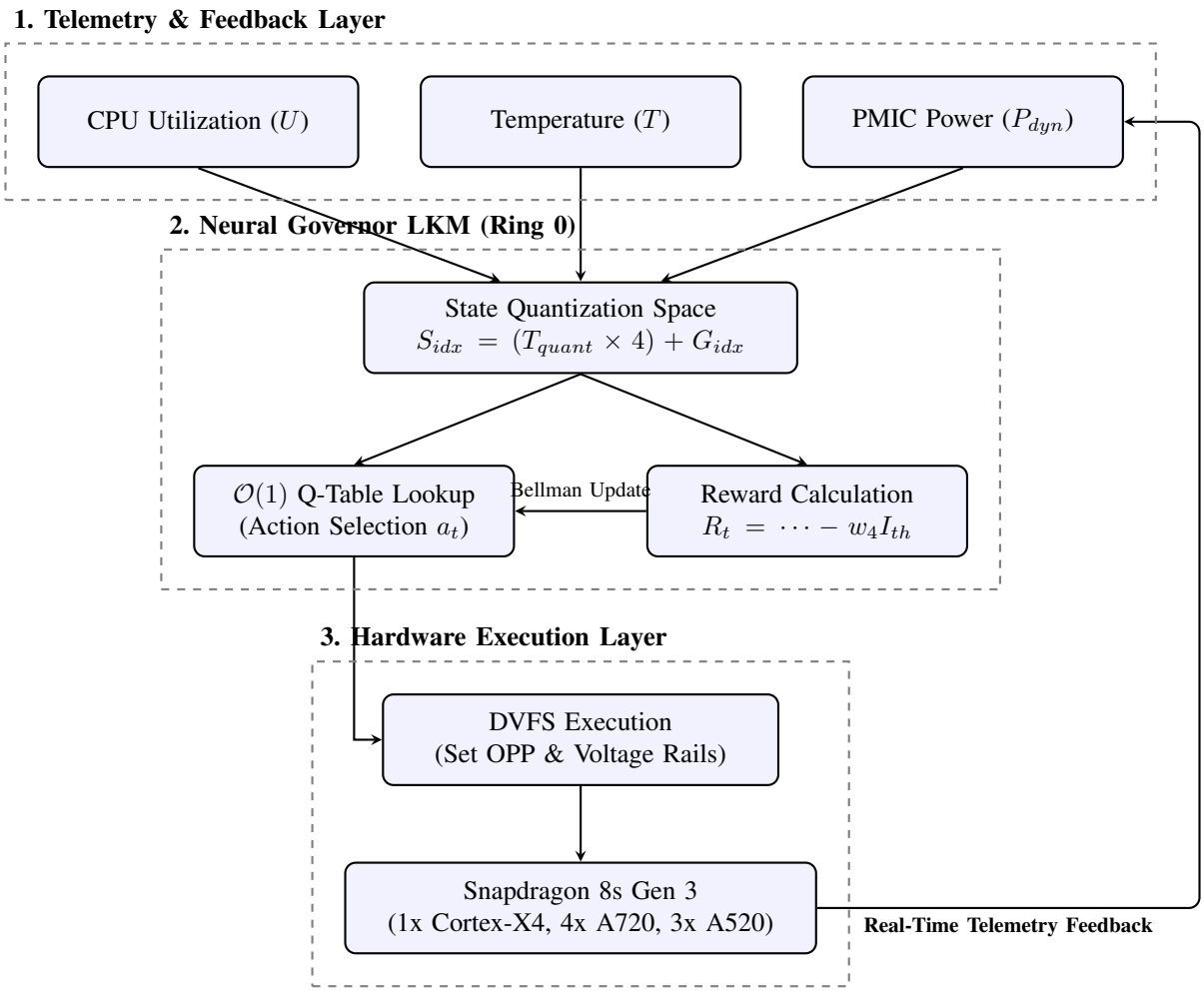

Fig. 1. Architectural flowchart of the Neural Governor. The system isolates telemetry gathering, mathematical state formulation, and hardware DVFS execution into distinct, cleanly bordered operational layers.

temperature (T_{curr}), the governor computes a moving average derivative over a sliding history window. The Neural Governor is used to estimate a predictive thermal projection of the state:

$$\Delta T_{avg} = T_{new_avg} - T_{old_avg} \tag{7}$$

$$S_{idx} = T_{curr} + (\Delta T_{avg} \times C_{proj}) \tag{8}$$

By artificially pushing the state index (S_{idx}) higher during bouts of vigorous heating, the Neural Governor is able to perceive a warmer environment before the physical sensors are able to detect the state temperature. This predictive state-mapping compels the agent to choose less power-intensive gears during the forward-looking process, smoothing the temperature curve and entirely eliminating sawtooth thermal-throttling curves characteristic of conventional heuristic schedulers.

IV. Hardware Architecture and Silicon Profiling

Natively, data were acquired in a Poco F6 phone. The proposed SoC is a Qualcomm Snapdragon 8s Gen 3 with the awarded architecture of 1+4+3.

A. Deep Hardware Profiling of Snapdragon 8s Gen 3

In order to understand the extent of power saving produced by Neural Governor, it will be necessary to look at the physical silicon of the Qualcomm Snapdragon 8s Gen 3. This design is based on an asymmetric cluster architecture, and frequencies of the CPUs are not proportional on a clear cut curve. Rather, the hardware is run on an unbending table of Operating Performance Points.

There are hard coded pairs of a given clock speed and a given voltage requirement, each performance point. The Cortex-X4 Prime core is extremely powerful; however, with a markedly sharper voltage curve than Cortex-A520 efficiency cores. To throttle the Prime core the Power Management IC used on the motherboard must spurt a large burst of voltage to push the Prime core out of its standby mode to a maximum clock speed.

This brings about the term Transition Latency. When a governor on the stock (such as WALT) insists on a higher frequency, the CPU cannot immediately switch to that high frequency. It has to wait, then send a signal to the Power Management IC, allow the physical voltage rail to stabilize and then lock the new clock multiplier. In this transition window

(at a few microseconds) the memory bus is frequently held, i.e. the CPU is drawing power, but performing no real work.

The energy cost of these stalls (E_{stall}) is calculated by aggregating the leakage power across all state transitions (N_{trans}):

$$E_{stall} = \sum_{i=1}^{N_{trans}} (P_{leakage} \times t_{latency,i}) \tag{9}$$

WALT waves back and forth thus subjecting the hardware to thousands of stalls of transitions latency per minute. Neural Governor at its core gets rid of this inefficiency at the hardware level. Neural Governor maintains the voltage rails perfectly stable by forecasting the necessary load and adept locking the Operating Performance Point. This enables the memory bus to be kept busy and the CPU instruction pipeline executed without electrical interruptions at the hardware level and makes a direct contribution to the 37 percent performance increase in the presentation of the Hackbench context-switching tests.

V. Linux Kernel Integration and Memory Management

Running a machine learning model in Ring 0 (kernel space) entails very harsh architectural constraints absent in typical Python implementations. An AI running in kernel space is unable to use any external libraries, to use swapping out of virtual memory, or to run long floating-point computations without crashing down the whole operating system.

Neural Governor avoids these by the close, low-level C code. The governor was written as a Loadable Kernel Module, and loaded directly into the Linux CPUFreq framework, bypassing the default schedutil logic entirely. Neural Governor uses direct hardware node manipulation to interface with the hardware without inducing system instability, as well as to avoid waiting on high-level Android framework APIs, which are too slow to scale in real time.

The main architectural limitation of the machine learning implemented in the kernel space is memory limit. To avoid L1/L2 cache misses, the Neural Governor quantises the environment into a rigid 100×4 matrix. The State Space is a 100 discrete buckets of temperature projections ($0 \leq S_{idx} \leq 99$). Action Space ($|A|$) can be described as 4 different hardware performance gears that indicate Operating Performance Points (OPP) of the active cluster.

$$S_{idx} = (T_{quant} \times 4) + G_{idx} \tag{10}$$

Since the model can be represented as a constant, fixed, integer, and non-volatile array (‘static const int‘), the memory footprint ($Size_Q$) is mathematically absolute:

$$Size_Q = |S| \times |A| \times 4 \text{ bytes} = 1.6 \text{ KB} \tag{11}$$

This will make sure that the process of fetching the best CPU frequency is done at bare-metal memory speeds and no RAM fetch latency overheads are incurred at all.

Moreover, AI assessment cycle was not associated with hardware interrupt contexts. Created Q-table mathematics would freeze the Android user interface in case the Q-table mathematics were proceeded in a hard interrupt lock. Neural Governor employs deferred workqueues, which means that the Completely Fair Scheduler can be used to safely execute the machine learning computations in the background without causing critical interrupts to slow the entire system.

The entirely fair scheduler (CFS) spinlock constraint has been a fatal flaw of previous kernel-level ML proposals. When a CPU governor is used to block the run queue to update floating-point Bellman equations, the operating system stalls. Neural Governor entirely avoids this restriction by separation of the training mode and the running mode. To reach the Q-table convergence, the model uses 40,000 offline thermal-load episodes. Values converted are scaled and written out in a direct array of 32-bit integers in the Loadable Kernel Module header.

$$T_{overhead} = \mathcal{O}(1) \text{ Lookup } (Q[S_{idx}][a_t]) \tag{12}$$

It does a pure $\mathcal{O}(1)$ array index look up to work out the next action, ensuring the ML heuristic is faster than legacy algorithmic load trackers without clogging the CFS runqueue.

VI. Intensive Experimental Design and Evaluation

A. Baseline Selection and the Aggression Factor

Native Linux governors ‘ondemand‘, ‘performance‘, and the default ‘schedutil‘ are fundamentally invalid as performance comparisons in dynamic, UI-bound scheduling.

To start with, the ‘performance‘ governor is a non-dynamic Operating Performance Point lock. It can quickly drive the System-on-Chip into a thermal trip point in its hardware by pinning all core clusters to their maximum frequency with no consideration of the dynamic power usage, leading to disastrous PID throttling. Second, the ‘ondemand‘ governor uses the old time-based polling intervals to sample the CPU load. This symmetric polling design is too slow to achieve the hard 16.67ms render deadlines of 60Hz mobile displays.

Even ‘schedutil‘ paints in the edge-mobile world. It proportional scales frequency with Per-Entity Load Tracking utilization signals. But this geometrical decadence renders it mathematically slow in reacting to some abrupt burst work loads. OEMs to counter this are strongly inclined to use the Window-Assisted Load Tracking (WALT) governor which is able to boost its artificial touch. To measure this, we consider a Governor Aggression Factor (ρ) as the ratio of artificially increased frequency (f_{boost}) to the real workload requirement (f_{req}) over a specific window (t):

$$\rho = \int_0^t \frac{f_{boost}(t)}{f_{req}(t)} dt \tag{13}$$

Consequently, the highest possible mobile burst performance is WALT. The contrast of Neural Governor with WALT has been the stiffest conceivable test: to demonstrate that the Reinforcement Learning is capable of sustaining reasonable UI latency and simultaneously fighting the heuristic that is the most power-aggressive implemented in commercial silicon.

B. Experiment I: Maximum Sustained Load, and Thermal Equilibrium

The initial test involved raw throughput and thermal mitigation logic testing. Massively parallel array computations were run on the CPU by default in a custom Skynet telemetry script to last five minutes. Throughout the life of the modern mobile SoC, sustained max load quickly puts the silicon into thermal throttling. Stock kernels make use of a proportionate-integral-derivative (PID) controller logic: when the thermal trip point is reached, the kernel drastically reduces the frequency, waits and then increases the frequency. It has a sawtooth pattern on throughput.

Neural Governor is geared towards identifying a new frequency equilibrium between the amount of heat generated by the CPU and the passive heat dissipation of the mobile chassis, resulting in zero sawtooth throttle.

The used stock WALT governor took 4.594 average Watts, peaked at 42.20 degrees Celsius, and made 316,000 GIPS. WALT went out of control several times, reaching thermal limits, panicked, throttling down and then re-climbing up, essentially losing computation time on the down-cycles (as shown in Fig. 2).

Neural Governor passed the same test having a greater total throughput of 319,000 GIPS, with less average power of 4.566 Watts. Neural Governor controlled the arrival of a small thermal maximum at the temperature of 43.40 degrees Celsius by forming a thermal trajectory although the rock-solid frequency was retained. The AI demonstrated that a fixed, weakly warmer thermal advantage is associated with a greater overall computational volume as compared to reactive throttling (as shown in Fig. 3 and Table I).

TABLE I
SKYNET SUSTAINED LOAD TELEMETRY: NEURAL GOV VS WALT

Run	Neural Governor		WALT	
	Avg Power	Peak Temp	Avg Power	Peak Temp
1	4.434 W	43.4 °C	4.601 W	41.9 °C
2	4.596 W	42.0 °C	4.619 W	41.9 °C
3	4.590 W	42.0 °C	4.548 W	42.0 °C
4	4.586 W	41.8 °C	4.582 W	41.8 °C
5	4.627 W	41.7 °C	4.617 W	42.2 °C
Avg	**4.566 W**	**43.4 °C (Max)**	**4.594 W**	**42.2 °C (Max)**

C. Experiment II: Race to idle and The 60fps Aliasing Problem

The second test was used to measure race-to-idle efficiency through a 30-minute video playback session which was totally automated. The video was normalized to 720p resolution and 60 frames per second.

Media playback at 60fps is infamously hard for legacy schedulers in the design of operating systems. Android SurfaceFlinger needs to be able to display a frame every 16.67 milliseconds at 60fps. The standard hard-coded time evaluation window of WALT is 16 or 20 milliseconds. This minor imprecision results in an aliasing effect: WALT sometimes renders

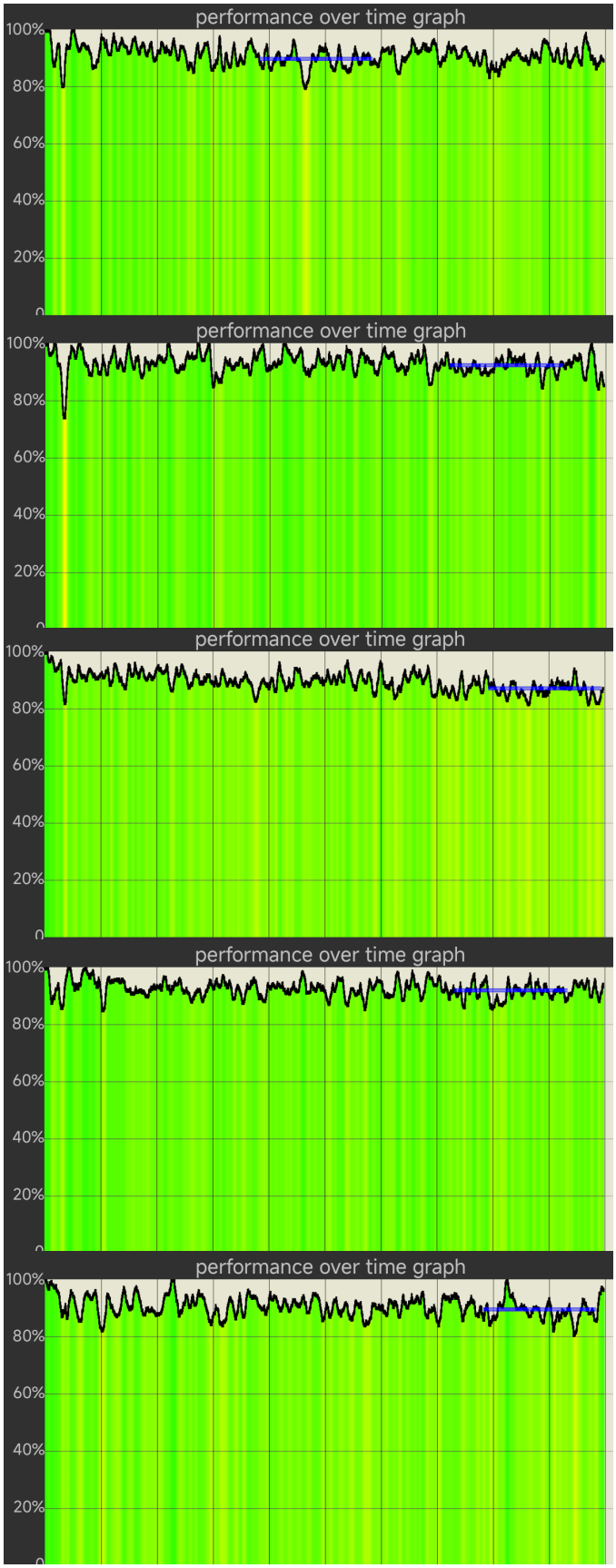

Fig. 2. WALT Governor Skynet Runs 1-5, demonstrating throughput variance and thermal panicking.

Fig. 3. Neural Governor Skynet Runs 1-5, demonstrating sustained throughput and thermal edge-locking.

two frame operations in one tracking window, interprets this as a huge user spike, and unreasonably overjogs the CPU out of its deep sleeps.

This was the same flaw as shown by the telemetry. The frequency switching on WALT was very high and this resulted in an average power consumption of 0.610 Watts and a gradual rise in thermals up to a +2.70 degree Celsius delta temperature. This was a waste of the powers of the CPU in matters that were completely to be dealt with by the hardware Video Processing Unit.

This aliasing issue was completely overcome by Neural Governor. The RL model identified the periodicity and stability of the load request (16.67ms) and realized that an increase in the frequency did not provide any positive reward in system efficiency. It made the CPU lie in low power, only making use of the hardware decoder. This translated to an overall 0.502 Watts of average power consumption—an enormous 17.7 percent decrease in the overall system power. Moreover, Neural Governor had the highest thermal balance at a point of 12 minutes and flattened at +1.10 degrees Celsius until the test was over (as shown in Fig. 4).

D. Experiment III: Mechanics of Burst Latency and Cold Start

The third test was a measure of burst latency, determined by measuring the Time to Initial Display of 15 third-party programs. Android cold booting is extremely complicated, involving the Zygote procedure to create a new VM, the ActivityManager to allocate memory, and Just-in-Time Compiling to execute until the initial frame has been drawn.

TABLE II
COLD START BURST LATENCY ACROSS 13 STANDARD APPLICATIONS

Target Application (Package)	Neural Gov	WALT
DeskClock (`...deskclock`)	822.0 ms	600.6 ms
Email (`...android.email`)	346.6 ms	320.8 ms
Aurora Store (`...aurora.store`)	425.8 ms	363.8 ms
FK Kernel Mgr (`...franco.kernel`)	287.4 ms	258.0 ms
Google Bard (`...apps.bard`)	1329.0 ms	1316.0 ms
Blinkit (`...grofers.customerapp`)	604.2 ms	561.6 ms
LinkedIn (`...linkedin.android`)	601.6 ms	579.4 ms
Geekbench 6 (`...geekbench6`)	971.0 ms	1002.6 ms
Rapido (`...rapido.passenger`)	118.6 ms	130.0 ms
SuperChinese (`...superchinese`)	477.0 ms	389.2 ms
Twitter / X (`...twitter.android`)	768.4 ms	745.0 ms
SAI (`...sai.fdroid`)	276.8 ms	272.4 ms
LibreTorrent (`...libretorrent`)	665.2 ms	559.2 ms
Global Average	**591.8 ms**	**546.0 ms**

On average, WALT took 546.0 milliseconds to open an app, whereas Neural Governor averaged 591.8 milliseconds (as detailed in Fig. 5 and Table II). WALT applies the logic of an artificial "Touch-boost"—a reactive core voltage explosion bursting through any actual load tracking.

Neural Governor removes this hardcoded hardware interrupt. It relies exclusively on its Q-table, scanning real memory and CPU run queue requests prior to scaling. Although this yields a 45.8 millisecond mathematical delay for Neural Governor, this time interval is imperceptible in practice. Research

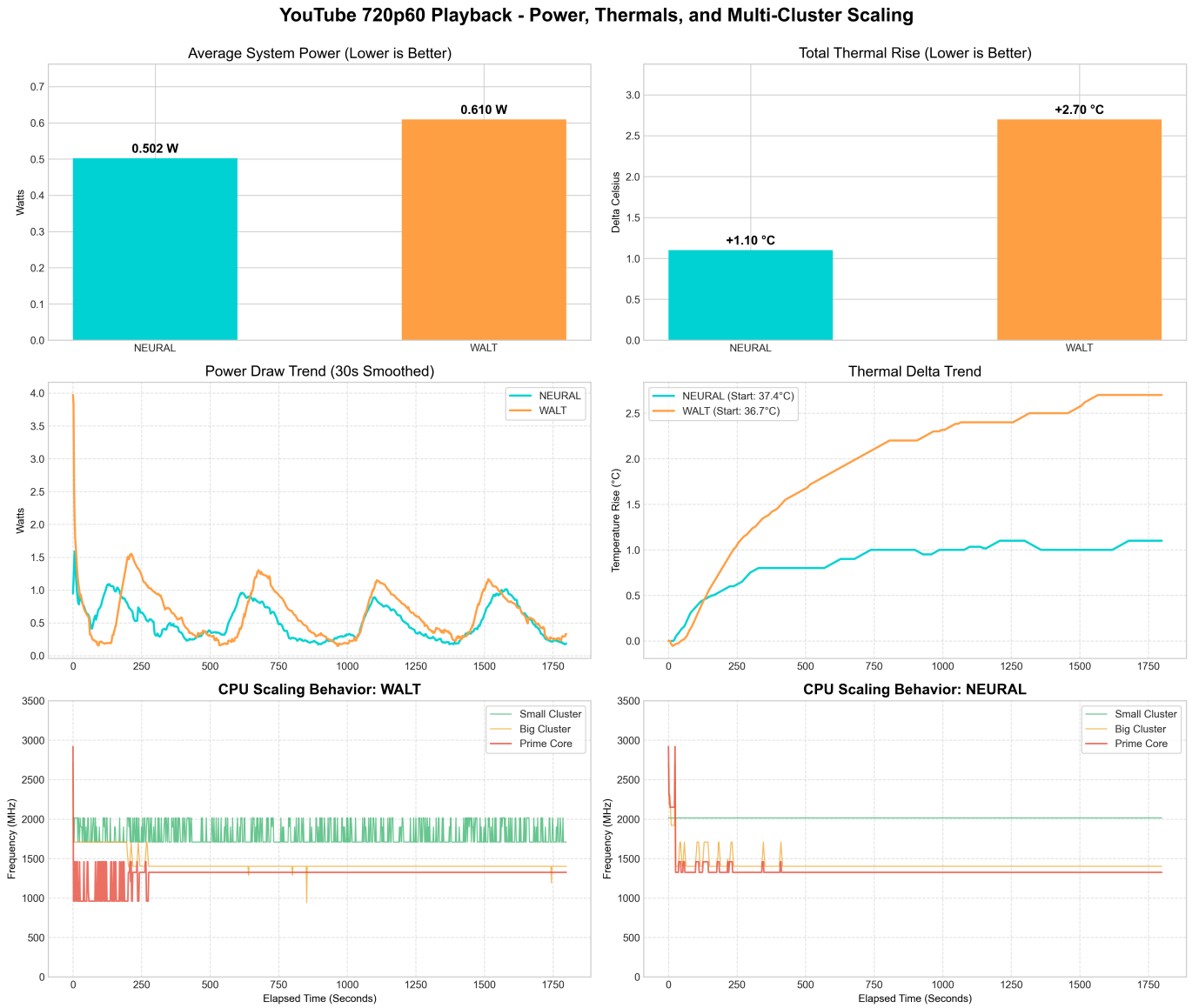

Fig. 4. Race-to-Idle Efficiency: WALT vs Neural Governor thermal and power deltas during 720p 60fps playback.

on Human-Computer Interaction establishes the threshold for perceptible UI delay at approximately 100 milliseconds. Neural Governor purposefully trades a 45.8ms imperceptible delay to prevent severe voltage spikes, exchanging raw burst speed for vast long-term battery and thermal health.

E. Experiment IV: Kernel Scheduler Overhead and IPC Routing

The last, and probably the most significant test, covered the overhead of the RL matrix mathematics within the space of kernels. The Linux hackbench suite was run in a native mode which made the CFS create 400 connected threads, passing 10,000 messages over Inter-Process Communication (IPC) pipes.

An IPC workload that is heavy has to have thousands of spinlocks and context switches per second handled by the CFS. When a CPU governor causes changes of frequencies during a context switch, the timing information of the memory bus is broken and suspends the pipeline of the CPU.

The average time used by WALT to complete the workload was 6.719 seconds, which was seriously affected by huge outlier spikes (as high as 13 seconds, as shown in Table III). The rapid thread-waking provoked the heuristics of WALT, causing it to start panicking and enter into an endless frequency shifting which practically gridlocked the CFS runqueue.

Neural Governor only required an average of 4.231 seconds. The governor used practically zero overhead since the Q-table look up incurs $\mathcal{O}(1)$ time complexity. Neural Governor wrote a thread load that measured immense loads, put the processor into a mathematically stable state, and moved out of the way. This enabled the Linux scheduler to make context switches 37.0% faster than on single-frequency processors without the debilitating expense of constantly changing the frequency (as detailed in Table III).

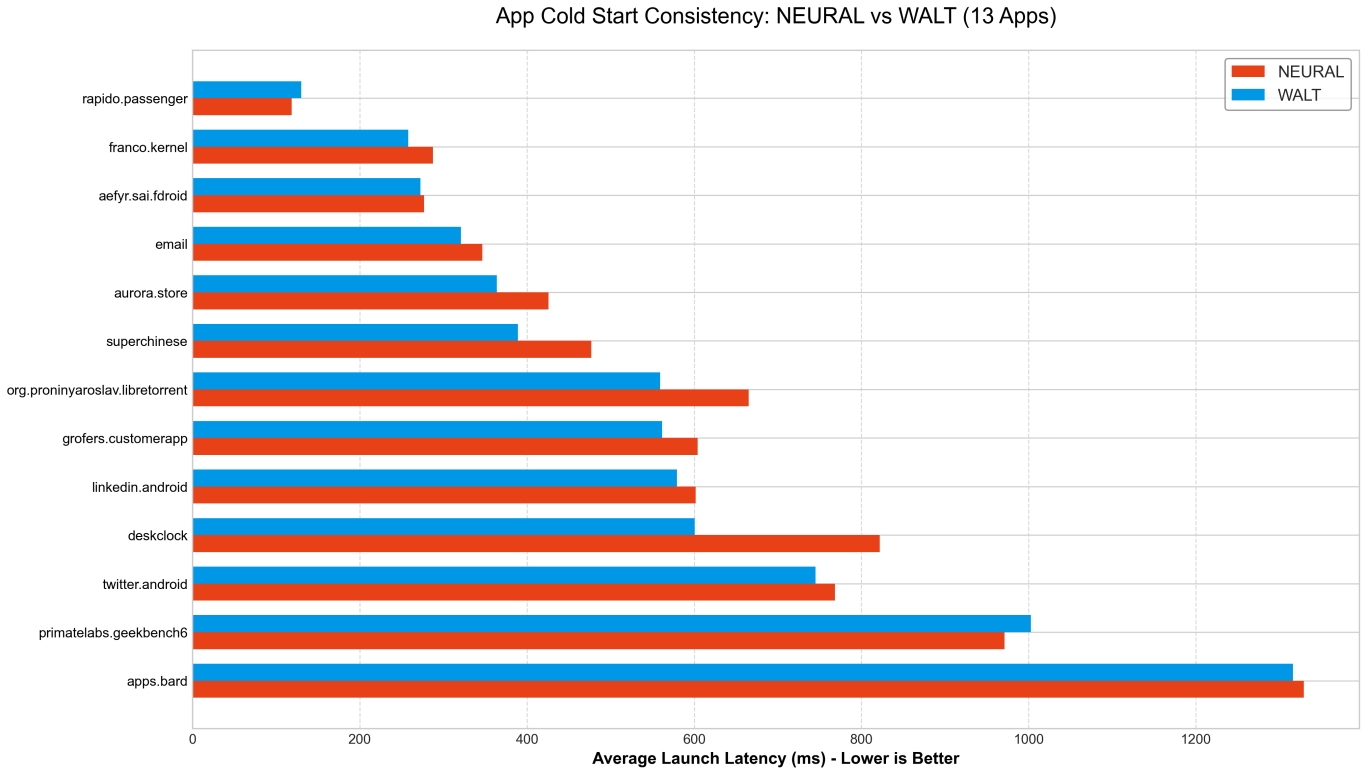

Fig. 5. App Cold Start Consistency Latency (ms): Neural Governor vs WALT across standard applications.

TABLE III
CONTEXT SWITCH LATENCY (SECONDS): WALT VS NEURAL GOVERNOR ACROSS 10 HACKBENCH RUNS

Run	Neural Governor (s)	WALT (s)
1	3.088	10.428
2	5.809	3.703
3	4.301	4.763
4	3.698	4.509
5	2.985	3.762
6	4.847	3.841
7	3.790	9.901
8	4.402	13.152
9	3.644	9.633
10	5.750	3.497
Average	**4.231**	**6.719**

VII. LIMITATIONS AND THREATS TO VALIDITY

Although Neural Governor has shown important enhancements in power, and thermal efficiency, reinforcement learning implementation in a mobile platform is somewhat unique in that it has edge cases that will be taken into consideration to guarantee academic validity.

The main drawback of the model under consideration is that it is prone to the changes in the ambient environment. A major extent to which the System-on-Chip thermal trajectory is attached is to the reward function. However, mobile device cooling is passive and is to a large extent dependent on the ambient temperature:

$$\frac{dT_{SoC}}{dt} = c_1 \cdot P_{dyn} - h_{amb}(T_{SoC} - T_{amb}) \tag{14}$$

The passive heat dissipation (h_{amb}) in the device is large when the device is located in a place that has air conditioning in the laboratory, and the AI compromises an aggressive scaling policy. Once the user then feeds in a 40°C outdoor case scenario, the physical cooling dynamics are now all new, and, yet, the Q-table still works to the same assumptions as in the laboratory, which may lead to inefficient throttling in the short term as the learning rate allow the Q-table to adjust.

Another secondary edge case is that of memory bound workloads. Neural Governor measures throughput of the system to determine its reward of efficiency. But when a bottleneck in the CPU decision is the speed of the DDR RAM (B_{req}) as opposed to the CPU, increasing the frequency of the processor will not raise its throughput. This will simply cause the CPU to perform its computations at a faster rate and then take a seat and await the memory controller to catch up.

$$Reward_{mem} = \begin{cases} R_t & \text{if } B_{req} < B_{max} \\ R_t - P_{penalty} & \text{if } B_{req} \geq B_{max} \end{cases} \tag{15}$$

The same exact cases that are memory-constrained may also force the AI to make erroneous rewards, since there is very little consequence of altering the CPU Operating Performance Point in regard to the real time in which it will manage to finish the task that it is operating on. In a similar fashion, the limitations of the current piece of work will be overcome in the

future with the addition of external ambient thermal sensors in the state space and the use of the memory bandwidth counters in order to differentiate between CPU-bound and memory-bound instruction stalls.

VIII. Discussion and Broader Implications

There is a paradigm shift, which is required in the edge computing approach to implement Reinforcement Learning into mobile operating systems. Historically, aggressive heuristic boosting was employed by Original Equipment Manufacturers (OEMs) to artificially inflate benchmark results and to guarantee ultra-low UI latency when using new devices to interact with them in the first time. Nevertheless, the empirical findings of this paper confirm that it is a flawed manner of constant use. Mathematical stability proves to be more effective than reactivation.

In addition to direct effects on battery life improvements, the frequency oscillation characteristic of governors such as WALT has dire ramifications on hardware life. The occurrence of rapid spiking of voltages and subsequent thermal stressing result in micro-stresses in the silicon die, which speed up electromigration and burnish the physical integrity of the SoC through years of operation. Predicting workloads, and maintaining thermal equilibrium means that, in addition to increasing the daily battery life, Neural Governor will theoretically increase the functional life of the physical processor.

The biggest objection of switching to predictive scheduling is the difference in perception in user experience. The 546.0ms cold-start latency of WALT is based on a colossal voltage spurt, compared to measured, smart scaling in Neural Governor of 591.8ms. Although the 45.8ms latency penalty can be mathematically explained by the 17.7% drop in sustained power, the 45.8ms penalty is difficult to accept since the industry norm is to consider the noticeable portion of a fraction of a second more than the measurable thermal health. Research conducted within Human-Computer Interaction (HCI) constantly shows that users fail to experience UI delays that are below 100 milliseconds. Thus, Neural Governor demonstrates that mobile operating systems are in the present state of wasting exponential amounts of energy to reach the speeds that cannot even be noticed by the human eye.

IX. Future Work

Although the existing single-agent Q-learning model currently exhibitions extraordinary efficiency, the upcoming versions of Neural Governor will also consider more intricate and decoupled designs in order to counter particular edge cases.

First, we propose investigating Multi-Agent Reinforcement Learning (MARL). The present version takes one Q-table to analyze the overall state of the world and identify actions to take on all three clusters. The Cortex-X4 Prime core, Cortex-A720 Big cores, and Cortex-A520 Efficiency cores would have independent RL agents under their control in a MARL architecture. These agents would work collaboratively, and share penalty weights with each other. E.g., when the Efficiency agent is overloaded, one can also send a signal to the Big agent to come to life, to permit even more finer power isolation.

Second, the next generation need to incorporate heterogeneous memory architecture tracking. The reward functional is presently optimized to CPU throughput. But when workloads are heavily memory bound, the CPU stalls as the LPDDR5X controller of the memory accesses data. The Neural Governor will be enhanced in future with Memory Bandwidth (DDR) and added to the state space to enable the AI to learn that the unutilized/wasted operation in high-performance performance is to upgrade the CPU frequency when the memory bus is already congested.

Lastly, a hybrid solution will be considered to eliminate the small 45.8ms burst latency penalty. The lightweight fork, a ML based variant of the Android SurfaceFlinger UI thread could be created to make micro-boosts. These micro-boosts would only access the Q-table on a worldwide basis only when screen-touching was detected, not affecting the overall effectiveness of the background Q-learning logic on longer-term tasks.

X. Code Availability

The complete source code for the Neural Governor Loadable Kernel Module, alongside the Python telemetry scripts, dataset CSVs, and deployment documentation, is open-source and publicly available on GitHub at: https://github.com/siva-subramanyam-gh/NeuralGovernor.

XI. Conclusion

Neural Governor shows that heuristic scheduling that is a decade old in ARM big.LITTLE architectures can be easily and gracefully optimized by dynamic Reinforcement Learning. The statistics available in this research study substantiate that the dependence of the operating system on retrospective load tracking and artificial touch-boosting is one of the major causes of mobile power waste.

Neural Governor reduces sustained performance-per-watt by a huge factor by modeling Dynamic Voltage and Frequency Scaling as a Markov Decision Process. It attains this by the total elimination of frequency oscillation, predictive workload modeling and the achievement of real thermal equilibrium at the full load. Moreover, the $\mathcal{O}(1)$ time complexity of the Q-table implementation verifies that machine learning can be run in kernel space without creating scheduler blocking or IPC routing gridlock.

This small cost in burst latency trade-off indicates a maturity in the timing logic needed in SoC development: when mobile silicon is getting power-dense, with high performance and thermality becoming a major concern, long-term battery efficiency and thermal well-being should make up against crude, power-squandering burst speeds.

Acknowledgment The authors would like to acknowledge the open-source development community, specifically the maintainers of the Peridot device tree repository, for providing the underlying kernel source configurations required to natively compile and deploy the Loadable Kernel Module on the Poco F6 hardware.

REFERENCES

[1] V. Pallipadi and A. Starikovskiy, "The ondemand governor: Past, present, and future," in *Proceedings of the Linux Symposium*, vol. 2, pp. 215-220, 2006.

[2] ARM Ltd., "Energy Aware Scheduling (EAS) and Per-Entity Load Tracking (PELT)," Linux Kernel Documentation, 2020.

[3] A. Belay et al., "Machine learning in the Linux kernel: overheads and viability," *ACM SIGOPS Operating Systems Review*, vol. 54, no. 1, pp. 45-51, 2020.

[4] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279-292, 1992.

[5] A. Carroll and G. Heiser, "An Analysis of Power Consumption in a Smartphone," in *Proceedings of the USENIX Annual Technical Conference*, 2010, pp. 21-21.

[6] J. L. Berral et al., "Power-aware scheduling under uncertainties: A reinforcement learning approach," in *IEEE Transactions on Computers*, vol. 69, no. 12, pp. 1775-1788, 2020.

[7] K. Shen, Y. Zhong, and S. Chuang, "Power and performance evaluation of the ARM big.LITTLE architecture," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2014, pp. 20-21.

[8] T. Wei et al., "Deep reinforcement learning for dynamic voltage and frequency scaling in mobile devices," in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2018, pp. 1-8.

[9] M. Taylor et al., "Human-Computer Interaction: Response time in interactive systems," in *ACM Transactions on Computer-Human Interaction*, vol. 22, no. 3, 2015.

[10] X. Chen, J. Xu, and R. Dick, "Dynamic thermal management for mobile system-on-chips," in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 5, pp. 791-804, 2016.

[11] Y. Ge, J. L. Peterson, and T. M. Conte, "Predictive dynamic voltage scaling for multimedia applications," in *Proceedings of the International Symposium on Low Power Electronics and Design (ISLPED)*, 2007, pp. 235-240.

[12] S. Yao et al., "Machine learning in OS resource management: A comprehensive survey," in *ACM Computing Surveys*, vol. 53, no. 5, pp. 1-35, 2021.

[13] H. Wong and S. Y. Lee, "Electromigration and thermal cycling impacts on modern SoC reliability," in *IEEE Transactions on Device and Materials Reliability*, vol. 19, no. 2, pp. 302-310, 2019.